Day 12 Artifact Decision: Comprehensive Analysis

Executive Summary

On Day 12, the challenge is to choose one artifact from four options: Evaluation Framework, Failure Taxonomy, Agent Pattern, or Prompt Test Harness. Each approach to scope definition reflects a different priority: systematic rigor, constraint-driven discovery, or immediate usability. This document synthesizes three perspectives—Perplexity, ChatGPT, and Claude—to help you choose based on your constraints and goals.

The Four Options

Option	Core Purpose	Outcome
Evaluation Framework	Systematically assess AI agent performance across multiple metrics	Reusable rubric for production testing
Failure Taxonomy	Classify and document failure modes	Reference documentation
Agent Pattern	Define architectural patterns for multi-step agents	Design template
Prompt Test Harness	Rapidly iterate and validate prompts against fixed inputs	Working tool + immediate feedback

---

Three Approaches Compared

1. Perplexity: Evaluation Framework

Philosophy: Assess before you scale. Poor performance detection wastes resources later.

Scope Definition:

- **Core Metrics:** Success rate, efficiency (steps to completion), safety (harm avoidance), goal alignment
- **Implementation:** Judge models or LangChain tracing
- **Example:** MLflow evaluating agent responses against ground truth in QA
- **Focus:** Single-agent eval only; 6 core metrics max; excludes deployment, cost optimization, multi-agent coordination

Key Steps:

1. Define goals aligned with business needs
2. Build test sets from production data
3. Run evals with LangSmith or MLflow
4. Iterate based on results

Best For: Teams building production agents; systematic rigor required; intent detection, tool use accuracy, output quality matter most.

---

2. ChatGPT: Minimal Harness (Constraint-First)

Philosophy: Constraint forces truth-telling. Most "good" prompts collapse under trivial perturbations. That discomfort is the point.

Non-Negotiable Hard Constraints:

- One task type only (e.g., reasoning over policy doc OR data extraction)
- One model family only
- Five prompts max
- Ten test cases max
- One output metric

Must Include:

- Fixed input set (same inputs every run)
- Prompt variant table (Prompt A → E)
- Scoring rubric (apply in <30 seconds per output)
- Comparison summary that makes a winner obvious

Explicitly Excludes:

- No UI
- No automation beyond copy-paste or simple script
- No "generalizable insights about prompting"
- No model comparison

Deliverable: Single Markdown/Notion page with:

- Task definition (3–4 lines)

- Test inputs
- Prompt variants
- Scoring rule
- Results table
- "What broke and why" paragraph

Best For: Individuals discovering where prompts actually break; discipline-first approach; discovering uncomfortable truths about prompt fragility.

---

3. Claude: Working Harness (Usable + Immediate)

Philosophy: Usable beats perfect. You need something you can click on today.

Scope Definition:

Included:

- Write prompt template once, test against many inputs
- Multiple test cases with optional expected outputs
- Pass/fail validation (substring matching)
- Direct API calls to Claude for actual evaluation
- Copy results to clipboard for analysis
- Live add/remove test cases

Excluded:

- Regex/fuzzy matching (substring only)
- Batch result export/CSV download
- Prompt versioning/history
- Cost tracking
- Custom grading rubrics
- Comparison between prompt versions

Deliverable: Interactive React app with:

- Prompt editor
- Test case management (add/remove)
- Real-time test execution

- Result visualization with pass/fail status
- Copy-to-clipboard for each result

Best For: Teams wanting to iterate today; interactive workflow preferred; immediate feedback cycle matters more than perfect metrics.

Where All Three Agree

1. **Ruthless scope definition is non-negotiable.** No sprawl into adjacent concerns (deployment, cost, multi-agent coordination, versioning, history).
2. **One task type only.** Mixing tasks makes validation meaningless.
3. **Small test set.** 5–10 test cases maximum. More creates false confidence; smaller forces you to think hard about edge cases.
4. **Scoring must be fast.** You can't spend 5 minutes per output. The rubric must apply in seconds.
5. **The point is discomfort.** All three approaches emphasize discovering what actually breaks, not validating what you already believe.
6. **Focus on single-agent, single-model evaluation.** Don't compare models or coordinate multi-agent behavior in this 60-minute artifact.

Key Tensions

UI vs. Discipline

- **ChatGPT says:** "No UI. Markdown table. Copy-paste." Forces rigor and prevents yak-shaving.
- **Claude says:** "Here's a working app." Trades some rigor for iteration speed and immediate usability.

Simplicity vs. Production-Ready

- **ChatGPT:** Markdown page, manual scoring, minimal tooling.
- **Claude:** Interactive app, but no batching, export, or versioning.
- **Perplexity:** MLflow/LangSmith integration; scales to production testing.

Discovery vs. Systematization

- **ChatGPT:** Find what breaks (discovery-first).
- **Claude:** Iterate on what works (iteration-first).

- **Perplexity:** Systematize all findings (framework-first).

Decision Framework

Choose based on your constraint:

If you have <30 minutes:

Go with ChatGPT's approach. Markdown + manual scoring. Discipline > tooling.

If you have 60 minutes and need to iterate today:

Use Claude's harness. A working tool beats a manifesto. You can click, run, copy, repeat.

If you're building for a team that will run 100+ tests:

Adopt Perplexity's framework thinking. Judge models, LangSmith/MLflow tracing, reusable metrics. Worth the setup overhead at scale.

If you're uncertain:

Start with ChatGPT's constraints, deliver Claude's tool. Constraints discipline your thinking; tool enables your iteration.

Implementation Guidance

Perplexity: Evaluation Framework

Time allocation (60 min):

- Define 3 core metrics (10 min)
- Build 10–15 test cases from production data (20 min)
- Set up LangSmith/MLflow integration (20 min)
- Run first eval, document results (10 min)

ChatGPT: Minimal Harness

Time allocation (60 min):

- Define task + business goal (10 min)
- Write 5 prompt variants (20 min)
- Create scoring rubric (10 min)

- Test manually, fill results table (15 min)
- Write "what broke" summary (5 min)

Claude: Working Harness

Time allocation (60 min):

- Deploy interactive app (5 min, already built)
- Define 1–2 prompt templates (15 min)
- Create 8–10 test cases (20 min)
- Run tests, iterate on prompts (15 min)
- Document findings (5 min)

The Real Insight

All three approaches are saying the same thing in different languages: **constraints → clarity → speed**.

- **Perplexity** constrains scope to single-agent, 6 metrics, production data.
- **ChatGPT** constrains to 5 prompts, 10 inputs, 1 metric, pure Markdown.
- **Claude** constrains to 1 task type, 1 model, substring matching, no versioning.

The constraint isn't a limitation—it's your forcing function. It prevents you from building a "general evaluation framework" (which fails) and forces you to build *something specific that actually works*.

Recommendation

For most teams on Day 12: Use Claude's harness with ChatGPT's constraints.

That means:

1. Pick ONE specific task (Claude's app lets you do this)
2. Write exactly 5 prompt variants (discipline from ChatGPT)
3. Use 10 fixed test cases (both agree on this)
4. Apply one simple scoring rule (pass/fail on substring match)
5. Document what broke (the uncomfortable truth)

This gives you:

- ✅ A tool you can use immediately (Claude)

- ✅ Rigor from constraints (ChatGPT)
- ✅ Repeatability (Perplexity's thinking applied at smaller scale)

Next Steps

1. **Decide your time budget.** <30 min? Use ChatGPT's Markdown approach. 60 min? Use Claude's app.
2. **Pick your task ruthlessly.** Not "summarization." Specifically: "Summarize policy documents under 500 words while preserving actionable items."
3. **Write 5 prompts.** Include 1 baseline + 4 variants (more precise instructions, different tone, different structure, etc.).
4. **Create 10 test cases.** Use real examples if possible. Mix easy + hard cases.
5. **Run tests.** Document what broke.
6. **Share the "what broke" paragraph.** That's where the value is. The table is just evidence.

Conclusion

The artifact you choose on Day 12 reflects what you optimize for:

- **Evaluation Framework:** Robustness and repeatability across many scenarios
- **Minimal Harness:** Discipline and uncomfortable truth-telling
- **Working Harness:** Iteration speed and immediate feedback

None is "wrong." But all three insist on the same thing: ruthless scope definition, because 60 minutes is not enough for both precision *and* breadth.

Choose the one that removes your biggest friction today. Then do it completely. Incompleteness is worse than simplicity.